TaskBoard: from git commit to a monitored Kubernetes cluster

DevOps Final Capstone · Ramneek Singh · October 2026

React · FastAPI · PostgreSQL · GitHub Actions · Trivy · GHCR · Terraform · EKS · Helm · Argo CD · Prometheus · Grafana

A task board shipped like a production service

Teams create tasks, set priority and assignee, and move them through TODO → IN PROGRESS → DONE. Every change travels the same path: tests, image build, security scan, registry, GitOps deploy, metrics.

6

task REST endpoints
+ health, ready, metrics

17

automated tests
13 pytest + 4 frontend

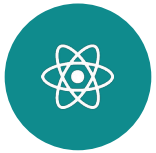
2

container images
scanned and SHA-tagged

59

AWS resources
created by Terraform

Tech stack



Application

React + Vite UI on Nginx (non-root), FastAPI, SQLAlchemy, Alembic, PostgreSQL 16



Quality

pytest on SQLite, Node test runner, Compose healthchecks



CI/CD

GitHub Actions, Trivy image gate, GHCR with commit-SHA tags



Infrastructure

Terraform AWS VPC and EKS modules, region ap-southeast-2



Platform

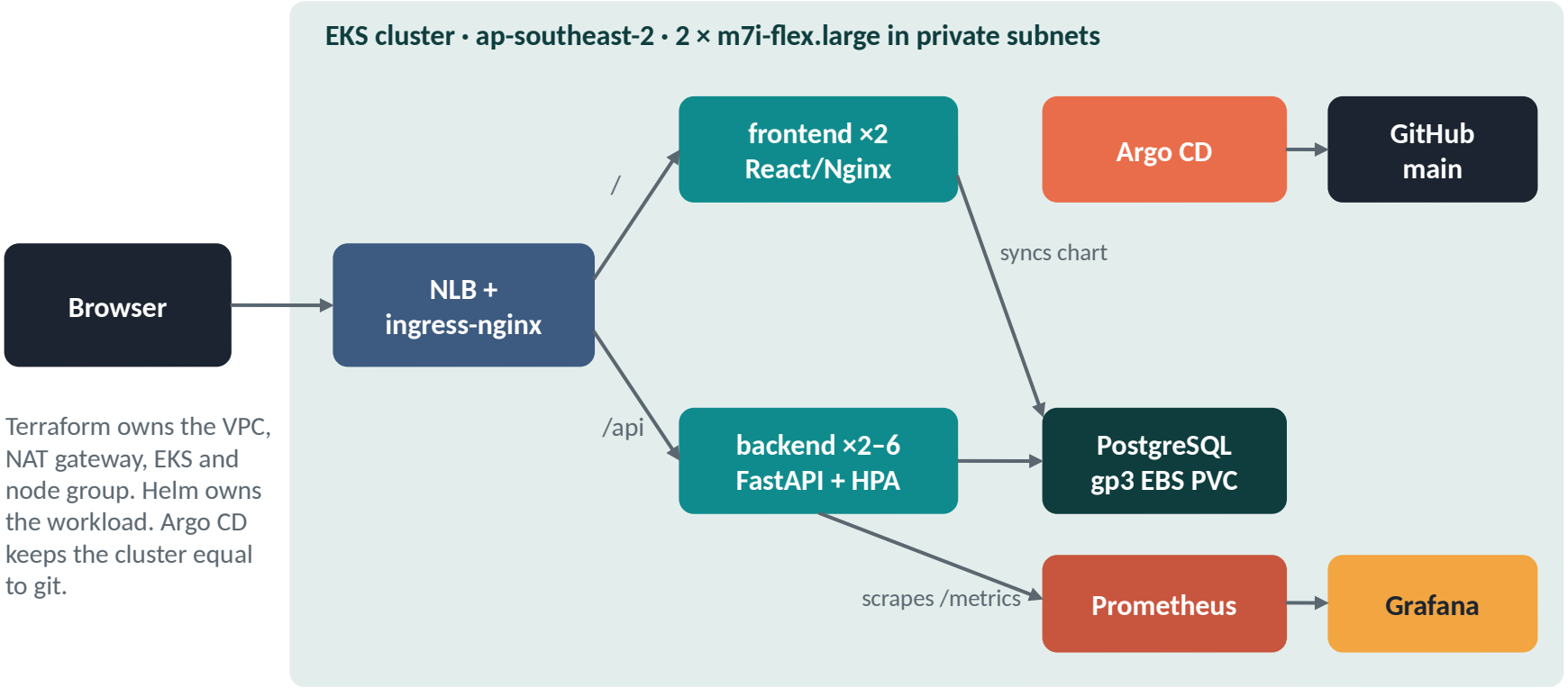
EKS 1.35, Helm chart, ingress-nginx, HPA, Metrics Server, Argo CD



Observability

kube-prometheus-stack: Prometheus Operator, ServiceMonitor, Grafana

Architecture on AWS



Terraform owns the VPC, NAT gateway, EKS and node group. Helm owns the workload. Argo CD keeps the cluster equal to git.

CI/CD: every push to main becomes a deploy



Push	Test	Build	Scan	Publish	GitOps	Sync
git push to main	pytest, npm test, Vite build	backend and frontend images	Trivy HIGH/CRITICAL fails the job	GHCR tagged with the commit SHA	bot writes SHA to values-eks.yaml	Argo CD rolls the cluster

Quality gates stop bad code before an image exists. Tests run first; a failure means no image, no scan, no tag.

Images are immutable. ghcr.io/ramneek3/taskboard/backend:<sha>; never latest, so every pod traces to a commit.

Deploy is a git commit. No cluster credentials live in GitHub; Argo CD pulls the change.

DevSecOps controls



Trivy gate

Both images scanned in CI; HIGH or CRITICAL with a fix fails the build. Current scans are clean after moving to Alpine bases and patched FastAPI/Starlette.



Non-root containers

API runs as uid 10001, Nginx as 101, Postgres as 70. runAsNonRoot is enforced in the pod spec.



No secrets in git

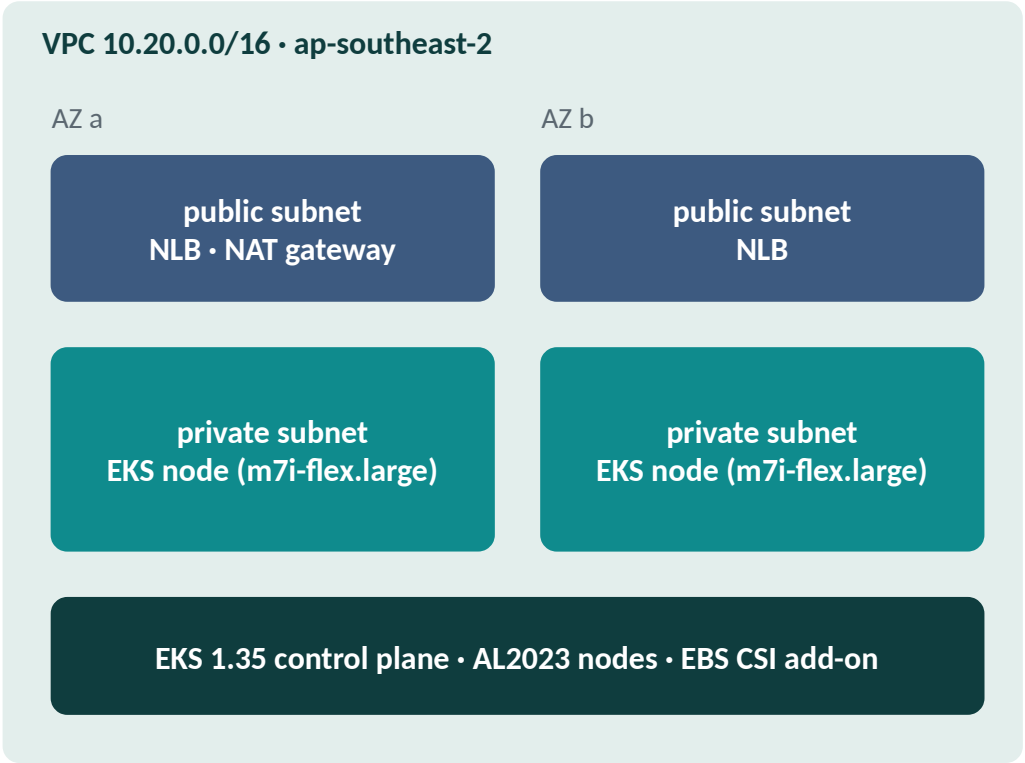
DB and Grafana passwords are generated into Kubernetes Secrets at install time. .env, tfvars and tfstate are gitignored.



Least exposure

Nodes in private subnets behind a NAT gateway; only the load balancer is public. EKS secrets are KMS-encrypted.

Terraform: AWS network and EKS as code



terraform init → plan → apply → destroy

- Community modules: terraform-aws-modules/vpc 5.8.1 and eks 20.37.1
- Plan: 59 resources to add, 0 to change
- Region pinned to ap-southeast-2: the account's organization SCP denies every other region
- tfvars.example committed; real tfvars and state never are
- About $0.40/hour while up; destroyed after the demo

Kubernetes, Helm and Argo CD

Deployments	frontend ×2, backend ×2 with startup, readiness (/ready) and liveness (/health) probes
Services	ClusterIP for frontend, backend and Postgres
Ingress	ingress-nginx on an AWS NLB: / → frontend, /api → backend
HPA	backend 2–6 replicas at 70% CPU, fed by Metrics Server
Storage	Postgres on a 5 GiB encrypted gp3 PVC
Monitoring	ServiceMonitor + Grafana dashboard ConfigMap in the chart

GitOps with Argo CD

- Application taskboard watches helm/taskboard on main with values-eks.yaml
- Automated sync with prune and self-heal
- helm upgrade --install for the first release; Argo CD keeps it in sync afterwards

Observability: Prometheus and Grafana



Traffic

Requests per second, by status code

Errors

Rate of 5xx responses from the API

Latency

p50 and p95 request duration

Business

Tasks by status; created, updated and deleted per second

Dashboard "TaskBoard API" ships inside the Helm chart, so Grafana picks it up as soon as the release is installed.

Troubleshooting drills



ImagePullBackOff

Break

troubleshooting/broken-image.yaml references a tag that does not exist.

Diagnose and fix

kubectl describe pod → Events show "manifest unknown" → fix the image tag.



Service with no endpoints

Break

troubleshooting/broken-service.yaml selects a label no pod has.

Diagnose and fix

kubectl get endpoints is empty → compare selector with pod labels → fix the selector.

Key learnings

01 Read the platform, not the tutorial

EKS 1.35 ships no Amazon Linux 2 node images; the node group had to move to AL2023.

02 Storage is not free on EKS

Without the EBS CSI add-on and a gp3 StorageClass, the Postgres PVC stays Pending forever.

03 Org guardrails win

The org SCP allows only ap-southeast-2 and blocks OIDC providers; the Free plan rejects t3.medium, so nodes run m7i-flex.large.

04 Tokens have scopes

GITHUB_TOKEN can only publish packages under the repository name, so images live at ghcr.io/<owner>/<repo>/<image>.

05 Liveness ≠ readiness

/health stays cheap; /ready checks the database so traffic only reaches pods that can serve it.

06 Destroy is part of the plan

Load balancers and volumes created by Kubernetes must go before terraform destroy can delete the VPC.

Thank you

github.com/ramneek3/taskboard

Live walkthrough next: codebase → CI run → Terraform → Argo CD → kubectl → Prometheus → Grafana